Propuesta piloto — Evaluación U1 resiliente a la IA — LPR (Encuentro 9)

> **Convención** `z-` : propuesta no vinculante del docente. No reemplaza a `evaluacion-u1.md` hasta aprobación explícita. No toca el corpus regenerable de `output/` : si se aprueba, el paso a producción es vía canon (`prompt-plantilla-planificacion.md` + datos de la materia → render), no editando `output/` a mano. **Derivado de:** `z-informe-evaluacion-resiliente-ia-LPR.md` (propuestas A1–A4, B5, B6, D9, E10 aplicadas; E11–E12 y C7/C8 completos quedan para instancias posteriores — ver §12). **Equivale a tres documentos del canon:** Parte I = `evaluacion-u1.md` · Parte II = `evaluacion-u1-version-a.md` · Parte III = `evaluacion-u1-version-a-anexo-docente.md` .

PARTE I — Documento de la instancia

1. Identificación

Campo	Detalle
Instancia	Evaluación de la Unidad 1 — Encuentro 9 (equivalente resiliente, piloto)
Unidad evaluada	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Evaluar la capacidad de **leer, diagnosticar, completar y extender** un programa de consola que combine entrada, conversión de tipos, condicionales y bucles — el trabajo real de un programador junior — y de **defender en vivo** lo producido.
Destinatarios	Todo el curso
Duración teórica	120 minutos (2 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	Este documento (Partes I, II y III)

2. Estructura del encuentro (120 min)

Momento	Tiempo	Qué ocurre
Apertura e instrucciones	10 min	Dinámica del encuentro, asignación de versiones por fila y **lectura de la política de IA de la prueba** (§8).
Prueba práctica individual	70 min	Cada estudiante trabaja sobre la base entregada: diagnosticar, completar, predecir y extender, con controles de avance por

Momento	Tiempo	Qué ocurre
		parte (§6).
Defensa del TP (rotación) + verificación oral de la prueba	30 min	Mientras el curso rinde, el docente defiende el TP con cada grupo (canon, sin cambios) e **intercala la verificación oral** con estudiantes que ya hicieron push (2–3 min por alumno).
Cierre y recogida	10 min	Últimos pushes, cierre del encuentro y anuncio de devolución para el encuentro 10.
Total	**120 min**	El bloque central y la rotación corren solapados, igual que en el canon.

3. Regla canónica de la instancia

La entrega del TP-U1 se verificó en el encuentro 8 (cierre de unidad). En este encuentro 9 se realizan la defensa individual del TP, la prueba práctica individual y la verificación oral de esa prueba. La devolución de todo ello se realiza al inicio del encuentro 10. Si un estudiante no completa la prueba en el tiempo asignado, entrega lo que tenga y se evalúa sobre lo entregado; no hay recuperación intra-encuentro.

4. Defensa individual del TP (modalidad — sin cambios respecto del canon)

Mientras el curso rinde la prueba, el docente se reúne por separado con cada grupo y verifica:

1. Que el repositorio existe en GitHub con la carpeta `tp-u1/`.
2. Que el archivo `kiosco.py` está presente y ejecutable.
3. Demo rápida: el grupo ejecuta el programa y muestra las opciones del menú.
4. Pregunta individual a cada integrante sobre una parte del código (p. ej., «¿qué hace este `while`?», «¿por qué usaron `try/except` acá?»).
5. Registro en planilla: cada objetivo del TP se marca como logrado / parcial / no logrado.

5. Alcance (sin cambios respecto del canon)

Incluido: variables y tipos (`int`, `float`, `str`), `input()` y `print()`, conversión con `int()` / `float()`, f-strings, condicionales `if` / `elif` / `else`, operadores de comparación y lógicos, bucle `while` con bandera, contadores y acumuladores, validación con `try/except ValueError`.

Excluido: colecciones (`list`, `tuple`, `set`, `dict`) como tema principal (el diccionario de productos se provee); funciones `def` como tema (se completan cuerpos, no se diseñan firmas nuevas); archivos; POO.

Objetivo mínimo: leer un programa de consola, detectar y corregir defectos de conversión/flujo explicando la causa, completar una función con validación `try/except` y extender el menú con una opción nueva.

6. Prueba práctica individual (diseño resiliente)

La prueba **no arranca de una página en blanco**: el docente entrega un `kiosco.py` incompleto con tres defectos sembrados. Puntaje total: 100 puntos (90 de la prueba + 10 de la defensa del TP). El enunciado

tiene cuatro partes con **control de avance** entre ellas:

- **Parte 1 — Diagnóstico de defectos (18 pts):** el programa corre pero hace cosas raras. Hay tres defectos; para cada uno: síntoma, causa en una línea y corrección. Reemplaza a los ítems conceptuales factuales del canon: mide comprensión, no memoria.
- **Parte 2 — Completar `register_sale()` (30 pts):** la función existe como esqueleto comentado; el alumno escribe el cuerpo cumpliendo la especificación 2a–2f del canon (validación, existencia, stock, descuento condicional, acumuladores).
- **Parte 3 — Predicción de salida (10 pts):** rastrear un fragmento corto (while + condicional + acumulador), predecir la salida exacta por escrito **antes de correr**, y explicar el rastreo.
- **Parte 4 — Extensión (18 pts):** agregar la opción «Recargar stock» al menú (transferencia de las mismas habilidades sobre código que ahora es propio).

Controles de avance: al cerrar cada parte, el estudiante corre la prueba de humo correspondiente y **pega la salida real como comentario** en el archivo (`# CONTROL PARTE N: ...`). Si la salida no coincide con lo esperado, el defecto sigue ahí: no conviene avanzar. El docente puede recorrer los controles en cualquier momento del bloque y la verificación oral los muestrea.

7. Criterios de calificación

Componente	Puntaje	Qué se observa
Parte 1 — Diagnóstico de defectos	18 pts	Por defecto (6 pts): ubicación exacta (2), causa explicada en palabras propias (2), corrección funcional (2).
Parte 2 — `register_sale()` completa	30 pts	2a código con validación (6) · 2b existencia (5) · 2c cantidad positiva (4) · 2d stock suficiente (5) · 2e descuento condicional (5) · 2f stock y acumuladores (5).
Parte 3 — Predicción de salida	10 pts	Salida exacta predicha (5) · rastreo justificado línea por línea (5).
Parte 4 — Recargar stock	18 pts	Menú actualizado (3) · código con `read_int` (3) · existencia (4) · cantidad positiva (3) · stock actualizado y mensaje con valor nuevo (5).
Convenciones y estructura	4 pts	Archivo único que ejecuta sin errores; f-strings con `:.2f` donde corresponde; identificadores en inglés `snake_case` ; mensajes en español accionables.
Verificación oral de la prueba	10 pts	Dos preguntas de muestreo (5 c/u) sobre lo que el alumno escribió; banco en el anexo, §G.
Defensa del TP	10 pts	Repositorio en orden, demo funcional, respuesta individual del integrante (canon, sin cambios).
Total	**100 pts**	Umbral de aprobación sin cambios: 60 pts.

8. Condiciones de resolución de la prueba (con política de IA explícita)

- **Individual:** cada estudiante resuelve su propia prueba en su computadora.

- **Prohibido en esta prueba:** asistentes de IA y chatbots (ChatGPT, Copilot y Copilot Chat, Gemini, Claude, DeepSeek y similares), webs que generan código, comunicación entre estudiantes, celular.
- **Permitido:** apuntes propios, material del aula virtual, documentación oficial de Python (sin asistentes integrados), autocompletado básico del editor.
- **Por qué está declarado:** la prueba mide lo que cada uno sabe hacer sin delegar. El uso supervisado de IA se evalúa en las instancias de trabajo grupal (TP), donde corresponde declaración de uso — no acá.
- **Convenciones obligatorias:** las de `input/materias/LPR/convenciones-tecnicas.md` (snake_case, f-strings, `if __name__`, sin persistencia, sin `except:` desnudo).
- **Entrega:** commit + push al repositorio grupal en la carpeta `evaluacion-u1/` antes de que termine el encuentro. Se espera al menos el commit de la base entregada y el commit final de entrega; los intermedios por parte son recomendados. Si no hay push, se evalúa el archivo local que el estudiante muestre.
- **Si se detecta uso de IA prohibida:** se aplica el régimen de convivencia del establecimiento y la prueba no se aprueba en esta instancia. *(Placeholder: el docente ajusta al régimen de la escuela.)*

9. Regla de equivalencia entre versiones

Las versiones A y B comparten la misma estructura, los mismos objetivos de aprendizaje y los mismos requisitos. Cambia exclusivamente el dominio (kiosco escolar ↔ biblioteca de aula) y los tokens de identificadores y datos. Los tres defectos sembrados y el esqueleto de `register_sale` son token-por-token, de modo que la versión B es derivable con `tools/generar-version-b.ps1` usando la tabla u1 existente. Faltan filas para los identificadores nuevos (`register_sale` , `restock_item` , `total_units` , `total_money`): agregarlas a la tabla o renombrar al vocabulario canónico de la tabla es decisión del docente en el paso a producción. La prosa natural de B recibe la pasada de lectura del docente (regla del generador).

10. Mecánica de asignación de versiones

Sin cambios respecto del canon: fila impar → versión A (kiosco), fila par → versión B (biblioteca). Se registra la versión en la planilla junto con el puntaje.

11. Devolución

Al inicio del encuentro 10: planilla con puntajes por criterio proyectada, comentario de errores frecuentes al grupo y devolución individual escrita. Quienes no alcancen 60 puntos rehacen el ejercicio en su versión corregida y lo presentan al inicio del encuentro 11 (canon, sin cambios). La verificación oral deja registro breve en planilla (dos preguntas y resultado), útil como evidencia de proceso.

12. Trazabilidad: qué cambió respecto del canon

Propuesta del informe	Aplicación en este piloto
A1 — código base	Material provisto: `kiosco.py` con 3 defectos sembrados + esqueleto de `register_sale` (Parte II).
A2 — justificación escrita	Partes 1 y 3 (causa en palabras propias; rastreo de la predicción).

Propuesta del informe	Aplicación en este piloto
A3 — entradas no previstas	Batería ampliada del anexo, §E.
A4 — checkpoints	Controles de avance con salida pegada como comentario, §6.
B5 — verificación oral de la prueba	10 pts, escalonada al hacer push; banco de muestreo en anexo §G.
B6 — pregunta de transferencia	Parte 4: extender el programa con una opción nueva.
D9 — ítems de comprensión	Partes 1 y 3 reemplazan a los conceptuales factuales.
E10 — política de IA declarada	§8, graduada y explícita.
E11 — instancia formativa «con IA»	Fuera del piloto: actividad de clase, no de evaluación.
E12 — declaración de uso de IA	No aplica dentro de una prueba presencial sin IA; aplicar en la política del TP.
C7 — commits en rúbrica	La entrega mantiene commit + push; el patrón de commits como puntaje queda para la rúbrica de TP.
C8 — estándar U4 extendido	Parcial: la extensión de la Parte 4 es el escalón previo; modificar el TP propio del grupo es el diseño U4 (canon).

PARTE II — Versión A para el estudiante (kiosco escolar)

> Dominio de esta versión: **kiosco escolar** (venta de articulos, stock y descuentos). Duración: 70 minutos (bloque de resolución). Puntaje total: 100 puntos (90 de la prueba + 10 de la defensa del TP). Resolución individual, con computadora, sin celular. Las condiciones completas están en la Parte I, §8.

Antes de empezar

- Descargá del aula virtual (carpeta «Evaluación U1») el archivo `kiosco.py` que entrega el docente y guardalo en la carpeta `evaluacion-u1/` de tu repositorio grupal.
- Hacé el primer commit de la base: `git add .` y `git commit -m "evaluacion-u1: base entregada"`.
- El programa **tiene 3 defectos sembrados y 1 función incompleta**: tu trabajo son las 4 partes de abajo, en orden.
- Convenciones del curso: identificadores en inglés y `snake_case`; comentarios y mensajes al usuario en español; f-strings; sin `except:` desnudo; sin archivos ni persistencia.
- Al cerrar cada parte, pegá la salida real de la prueba de humo como comentario (`# CONTROL PARTE N: ...`).

- **En esta prueba no se permite usar IA ni chats** (política leída en la apertura). Documentación oficial de Python y apuntes propios, sí.

Objetivos de la prueba

1. Leer un programa que usa variables, condicionales y bucles, y encontrar por qué falla.
2. Completar una función con validación `try/except ValueError`.
3. Rastrear un fragmento y predecir su salida exacta.
4. Extender un menú con una opción nueva reutilizando lo que ya existe.

Parte 1 — Diagnóstico de defectos (18 puntos)

El programa corre pero **no se comporta como dice la batería de verificación de abajo**. Hay **3 defectos**. Para cada uno escribí al final del archivo (después del bloque `if __name__ == "__main__":`), como comentario:

```
# PARTE 1 - DIAGNOSTICO
# Defecto 1:
#   Sintoma: <que hice y que paso, con la opcion probada>
#   Causa: <por que falla, en una linea>
#   Correccion: <que cambie, con la linea exacta>
# Defecto 2: ...
# Defecto 3: ...
```

Por defecto	Pts
Ubicar el defecto (acción y línea)	2
Explicar la causa en una línea propia	2
Corregirlo y que quede funcionando	2
Total por defecto (× 3)	**18**

Ayuda: corregí y probá de a un defecto a la vez; los tres son independientes.

Parte 2 — Completar `register_sale()` (30 puntos)

La función está como esqueleto comentado en el archivo. Completala para que cumpla, en orden:

Ítem	Consigna	Pts
2a	Pedir un código de producto con `read_int()` (ya existe: valida con `try/except`).	6
2b	Verificar que el código exista en `products`. Si no: "Codigo invalido. No existe ese producto." y volver al menú.	5
2c	Pedir la cantidad y validar que sea entero positivo.	4

Ítem	Consigna	Pts
2d	Verificar stock suficiente. Si no: "Stock insuficiente. Stock actual: {stock}." y volver al menú.	5
2e	Si la cantidad supera las 5 unidades, aplicar 10 % de descuento al subtotal y mostrarlo detallado.	5
2f	Descontar del stock y acumular en `total_units` y `total_money`.	5
Total Parte 2		**30**

Parte 3 — Predicción de salida (10 puntos)

Sin correrlo todavía, escribí como comentario al final del archivo qué imprime exactamente este fragmento y por qué (rastreo línea por línea):

```
total = 0
n = 1
while n <= 4:
    if n % 2 == 0:
        total = total + n
    else:
        total = total + 1
    n = n + 1
print(f"Total: {total}")
```

Después corralo en un archivo aparte. Si tu predicción y la salida real difieren, agregá una línea explicando en qué ciclo se desvió tu rastreo.

Ítem	Pts
Predicción escrita antes de correr (salida exacta)	5
Rastreo justificado (o diferencia bien explicada)	5

Parte 4 — Recargar stock (18 puntos)

Agregá al menú la opción **«4. Recargar stock»** y corré **Salir a la opción 5**:

Ítem	Consigna	Pts
4a	Menú con la opción 4 nueva y Salir como 5; las cinco opciones funcionan.	3
4b	Pedir el código del producto con `read_int()`.	3
4c	Verificar que exista; si no, mensaje claro y volver al menú.	4
4d	Pedir la cantidad a recargar y validar que sea positiva.	3

Ítem	Consigna	Pts
4e	Sumar la cantidad al stock y mostrar un mensaje con el nombre del producto y su stock nuevo.	5
Total Parte 4		**18**

Recargar **no** suma a `total_units` ni a `total_money`: no es una venta.

Batería de verificación: comportamiento esperado

Prueba	Pedido	Salida esperada
Opción 1	Ver lista	Tabla con código, nombre, precio y stock de cada producto
Opción 3	Mostrar resumen	Unidades vendidas y dinero acumulado con 2 decimales
Opción 4 / 5	Salir	"Saludos!" y el programa termina
Opción inválida	"9"	"Opcion invalida. Intenta de nuevo." y vuelve al menú
Opción 2 (venta)	Código 101, cantidad 3	"Total de esta venta: $150.00"; stock de caramelos: 27
Opción 2 (cantidad > 5)	Código 101, cantidad 7	"Subtotal: $350.00", "Descuento (10%): $35.00", "Total: $315.00"
Opción 2 (stock insuf.)	Código 101, cantidad 999	"Stock insuficiente. Stock actual: ..."
Opción 2 (entrada no numérica)	"abc" en el código	"Eso no es un numero entero; intenta de nuevo." y vuelve a pedir

> Después de la Parte 4, Salir es la opción 5 y la opción 4 recarga stock.

Al terminar

1. Verificá que el archivo se ejecuta sin errores: `python kiosco.py`.
2. Hacé `git add .`, `git commit -m "evaluacion-u1: entrega"` y `git push`.
3. Quedá disponible para la verificación oral (2–3 minutos): te va a tocar mostrar y explicar partes de tu propio archivo.
4. Si no podés hacer push, avisá al docente antes de que termine el encuentro.

PARTE III — ANEXO DOCENTE (no entregar a estudiantes)

> Solución de referencia, ubicación de los defectos sembrados, batería ampliada con entradas no previstas, criterios de corrección ítem por ítem y guía de la verificación oral.

A. Código base que se entrega (`kiosco.py`)

```
# kiosco.py — Sistema de ventas del kiosco escolar
# Evaluacion U1 — Version A — BASE ENTREGADA POR EL DOCENTE
# El programa tiene 3 defectos sembrados y 1 funcion incompleta.

products = {
    101: {"name": "Caramelos", "price": 50.0, "stock": 30},
    102: {"name": "Chicles", "price": 25.0, "stock": 50},
    103: {"name": "Galletitas", "price": 100.0, "stock": 20},
    104: {"name": "Jugos", "price": 150.0, "stock": 15}
}

total_units = 0
total_money = 0.0


def show_products():
    # Muestra la lista de productos del kiosco
    print()
    print("=== PRODUCTOS DEL KIOSCO ===")
    print(f"{'Codigo':<8} {'Nombre':<15} {'Precio':<10} {'Stock':<8}")
    print("-" * 45)
    for code, data in products.items():
        print(f"{code:<8} {data['name']:<15} {data['price']:<10.2f} {data['stock']:
<8}")


def read_int(message):
    # Pide un numero entero con validacion try/except (spike 9.1)
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Eso no es un numero entero; intenta de nuevo.")


def register_sale():
    # INCOMPLETA — Parte 2 del enunciado.
    # Registra una venta: pide codigo y cantidad, valida, descuenta y acumula.
    # Pasos: 2a codigo validado / 2b existencia / 2c cantidad positiva /
    #        2d stock suficiente / 2e descuento condicional (mas de 5) /
    #        2f descontar stock y acumular en total_units y total_money.
    pass


def show_summary():
    # Muestra el resumen de ventas de la sesion
    print()
    print("=== RESUMEN DE VENTAS ===")
```

```
    print(f"Unidades vendidas: {total_units}")
    print(f"Dinero acumulado: $total_money")


if __name__ == "__main__":
    # Bloque de ejecucion principal (spike 9.10)
    print("=== SISTEMA DE VENTAS DEL KIOSCO ===")
    running = True

    while running:
        print()
        print("1. Ver productos")
        print("2. Registrar venta")
        print("3. Mostrar resumen")
        print("4. Salir")
        option = input("Elegi una opcion: ")

        if option == 1:
            show_products()
        elif option == "2":
            register_sale()
        elif option == "3":
            show_summary()
        elif option == "4":
            print("Saludos!")
        else:
            print("Opcion invalida. Intenta de nuevo.")
```

B. Defectos sembrados (guía rápida de corrección)

#	Ubicación	Síntoma observable	Por qué falla	Corrección	Relación convenciones §9
1	`if option == 1:` (menú, rama 1)	La opción 1 siempre responde "Opcion invalida"	`input()` devuelve `str`; `"1" == 1` es `False`	`if option == "1":` (o validar conversión si usa `int(option)`)	Olvidar el tipo de `input()`
2	`print(f"Dinero acumulado: $total_money")`	En el resumen se imprime el texto literal `$total_money`, no el valor	F-string sin llaves: no interpola	`print(f"Dinero acumulado: ${total_money:.2f}")`	Salida con f-strings
3	`elif option == "4": print("Saludos!")`	El programa dice "Saludos!" y vuelve al menú: no termina	La bandera nunca cambia: `while`	Agregar `running = False` en la rama	Bucle con bandera

#	Ubicación	Síntoma observable	Por qué falla	Corrección	Relación convenciones §9
			`running` sigue en `True`		

Los tres son independientes: corregir uno no revela ni tapa los otros.

C. Solución de referencia completa

```
# kiosco.py — Sistema de ventas del kiosco escolar
# Evaluacion U1 — Version A — SOLUCION DE REFERENCIA

products = {
    101: {"name": "Caramelos", "price": 50.0, "stock": 30},
    102: {"name": "Chicles", "price": 25.0, "stock": 50},
    103: {"name": "Galletitas", "price": 100.0, "stock": 20},
    104: {"name": "Jugos", "price": 150.0, "stock": 15}
}

total_units = 0
total_money = 0.0


def show_products():
    # Muestra la lista de productos del kiosco
    print()
    print("=== PRODUCTOS DEL KIOSCO ===")
    print(f"{'Codigo':<8} {'Nombre':<15} {'Precio':<10} {'Stock':<8}")
    print("-" * 45)
    for code, data in products.items():
        print(f"{code:<8} {data['name']:<15} {data['price']:<10.2f} {data['stock']:
<8}")


def read_int(message):
    # Pide un numero entero con validacion try/except (spike 9.1)
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Eso no es un numero entero; intenta de nuevo.")


def register_sale():
    # Registra una venta: pide codigo y cantidad, valida, descuenta y acumula
    global total_units, total_money

    print()
    print("=== REGISTRAR VENTA ===")
```

```
    # 2a: Pedir codigo de producto
    code = read_int("Codigo del producto: ")

    # 2b: Verificar que exista
    if code not in products:
        print("Codigo invalido. No existe ese producto.")
        return

    item = products[code]

    # 2c: Pedir cantidad y validar positiva
    qty = read_int("Cantidad a comprar: ")
    if qty <= 0:
        print("La cantidad debe ser un numero positivo.")
        return

    # 2d: Verificar stock suficiente
    if qty > item["stock"]:
        print(f"Stock insuficiente. Stock actual: {item['stock']}.")
        return

    # 2e: Descuento condicional
    subtotal = qty * item["price"]
    discount = 0.0
    if qty > 5:
        discount = subtotal * 0.10
    sale_total = subtotal - discount

    # 2f: Actualizar stock y acumuladores
    item["stock"] -= qty
    total_units += qty
    total_money += sale_total

    print(f"Subtotal: ${subtotal:.2f}")
    if discount > 0:
        print(f"Descuento (10%): ${discount:.2f}")
    print(f"Total de esta venta: ${sale_total:.2f}")


def restock_item():
    # Parte 4: agrega stock a un producto existente (no es una venta)
    print()
    print("=== RECARGAR STOCK ===")

    code = read_int("Codigo del producto: ")
    if code not in products:
        print("Codigo invalido. No existe ese producto.")
        return

    item = products[code]
    qty = read_int("Cantidad a recargar: ")
    if qty <= 0:
        print("La cantidad debe ser un numero positivo.")
        return
```

```
    item["stock"] += qty
    print(f"Stock actualizado. Nuevo stock de {item['name']}: {item['stock']}.")


def show_summary():
    # Muestra el resumen de ventas de la sesion
    print()
    print("=== RESUMEN DE VENTAS ===")
    print(f"Unidades vendidas: {total_units}")
    print(f"Dinero acumulado: ${total_money:.2f}")


if __name__ == "__main__":
    print("=== SISTEMA DE VENTAS DEL KIOSCO ===")
    running = True

    while running:
        print()
        print("1. Ver productos")
        print("2. Registrar venta")
        print("3. Mostrar resumen")
        print("4. Recargar stock")
        print("5. Salir")
        option = input("Elegi una opcion: ")

        if option == "1":
            show_products()
        elif option == "2":
            register_sale()
        elif option == "3":
            show_summary()
        elif option == "4":
            restock_item()
        elif option == "5":
            running = False
            print("Saludos!")
        else:
            print("Opcion invalida. Intenta de nuevo.")
```

Aceptaciones válidas menores: nombres de funciones distintos; corrección del defecto 1 vía `int(option)` protegida con `try/except` (si convierte sin proteger, se rompe con entrada no numérica: ver §F); bandera reemplazada por `while True` + `break` (canon ya lo aceptaba); mensajes con redacción distinta pero accionable.

D. Parte 3 — salida esperada y rastreo

Salida exacta: `Total: 8`.

Rastreo esperado: `n=1` impar → `total=1`; `n=2` par → `total=3`; `n=3` impar → `total=4`; `n=4` par → `total=8`; con `n=5` el `while` corta. Respuestas frecuentes incorrectas: `10` (sumar todos los n), `4` (solo pares), `6` (pares + un solo impar). Si la predicción fue incorrecta pero al correr explica con precisión en qué ciclo se desvió, puntúa el rastreo completo.

E. Salidas de referencia y entradas no previstas

Pruebas canónicas (las del enunciado):

Prueba	Entrada	Salida esperada
Opción 1	`1`	Tabla de 4 productos con código, nombre, precio y stock
Opción 2 venta simple	`2`, `101`, `3`	"Total de esta venta: $150.00"; stock de caramelos: 27
Opción 2 con descuento	`2`, `101`, `7`	"Subtotal: $350.00", "Descuento (10%): $35.00", "Total: $315.00"
Opción 3	`3`	"Unidades vendidas: ..." y "Dinero acumulado: $..." (valor numérico, no literal)
Salir	`5` (o `4` antes de la Parte 4)	"Saludos!" y termina
Opción inválida	`9`	"Opcion invalida. Intenta de nuevo." y vuelve al menú

Entradas no previstas (validación en vivo del docente — evalúan comportamiento, no lectura de código):

Prueba	Entrada	Comportamiento esperado
Venta: código inexistente	`2`, `999`	"Codigo invalido. No existe ese producto." y vuelve al menú (sin KeyError)
Venta: código con espacios	`2`, `101`	`int()` tolera los bordes: la venta procede normal (no es defecto)
Venta: cantidad 0	`2`, `101`, `0`	"La cantidad debe ser un numero positivo."
Venta: cantidad negativa	`2`, `101`, `-3`	"La cantidad debe ser un numero positivo."
Borde del descuento: exactamente 5	`2`, `101`, `5`	Sin descuento ("supera las 5")
Borde del descuento: 6	`2`, `101`, `6`	Con descuento
Venta con letra en cantidad	`2`, `101`, `abc`	"Eso no es un numero entero..." y reintento
Recarga: producto inexistente	`4`, `999`	"Codigo invalido..." y vuelve al menú
Recarga: cantidad negativa	`4`, `101`, `-5`	"La cantidad debe ser un numero positivo."
Recarga válida no suma a ventas	`4`, `101`, `10` → `3`	El stock sube a 40 (o el que quede) y `total_money` NO cambia
Dos ventas seguidas	dos ventas válidas → `3`	El resumen acumula ambas (control de acumuladores)

Prueba	Entrada	Comportamiento esperado
Menú con texto	`abc` en el menú	"Opcion invalida..." sin crash

F. Criterios de corrección ítem por ítem

Ítem	Pts	Qué puntúa	Qué no puntúa	Errores previstos
1a — Defecto 1 (menú int/str)	6	Ubicar rama 1 (2) · causa `input()` → `str` (2) · corrección `== "1"` o `int(option)` protegido (2)	Justificación genérica	`int(option)` sin `try/except`: crash con "abc" en el menú → descontar 2
1b — Defecto 2 (f-string)	6	Ubicar `show_summary` (2) · causa sin llaves (2) · `f"...{total_money:.2f}"` (2)	Formato exacto del mensaje	Corrige pero sin `:.2f` → descontar 1
1c — Defecto 3 (bandera)	6	Ubicar rama Salir (2) · causa bandera sin cambiar (2) · `running = False` o `break` (2)	Uso de `sys.exit()`	`sys.exit()` → descontar 2 (canon §7 del anexo original)
2a — Código con `read_int`	6	Usa `read_int` o equivalente con `try/except ValueError` y reintento	Mensaje exacto	Sin validación → 0; sin reintento → descontar 3
2b — Existencia	5	`if code not in products` o `.get()`	Mensaje exacto	Sin chequeo (KeyError con 999) → 0
2c — Cantidad positiva	4	Entero validado y `qty <= 0` rechazado	Orden relativo de chequeos	No valida positivo → descontar 3
2d — Stock suficiente	5	`qty > stock` con mensaje y retorno	Mensaje exacto	Descuenta sin verificar (stock negativo) → 0
2e — Descuento condicional	5	`qty > 5` aplica 10 % al subtotal, mostrado detallado	Redondeo	Descuento en 5 exacto → descontar 3; sin descuento con 6+ → 0
2f — Stock y acumuladores	5	`stock -= qty`, `total_units +=`, `total_money +=`	Nombres de variables	Sin stock → -2; sin acumuladores → -3
3a — Predicción exacta	5	"Total: 8"	Procedimiento	10 / 4 / 6 → 0–2 según rastreo
3b — Rastreo	5	Aporte por ciclo o regla par/impar bien identificada	Extensión	Predicción mal pero diferencia bien explicada → hasta 4

Ítem	Pts	Qué puntúa	Qué no puntúa	Errores previstos
4a — Menú actualizado	3	Opción 4 nueva, Salir en 5, las cinco funcionan	Texto de opciones	Salir sigue en 4 → descontar 2
4b — Código con `read_int`	3	Reutiliza la validación existente	—	`input()` pelado sin validación → 0–1
4c — Existencia en recarga	4	Chequeo `in` con mensaje	Mensaje exacto	Sin chequeo → 0
4d — Cantidad positiva	3	Rechaza 0 y negativos	—	Sin validar → 0
4e — Stock y mensaje nuevo	5	`stock += qty` y mensaje con nombre y valor nuevo	Formato	Suma también a `total_money` → 0 (confunde venta con recarga)
Convenciones	4	Ejecuta sin errores (1) · f-strings y `:.2f` (1) · snake_case inglés (1) · mensajes en español accionables (1)	Estilo personal	—
Verificación oral	10	Dos preguntas del banco §G (5 c/u)	—	Ver §G para la escala
Defensa del TP	10	Canon, sin cambios	—	—
Total	**100**			

G. Guía de la verificación oral (banco de muestreo)

Modalidad: 2–3 minutos por alumno al terminar el push (o al inicio del bloque de cierre). El docente elige **2 preguntas**: al menos una de las partes 2 o 4 (código que el alumno escribió) y al menos una de las partes 1 o 3 (lo que diagnosticó o rastreó). Escala por pregunta: **5** responde y muestra en su código sin ayuda · **3** responde con ayuda mínima (una pista) · **0** no encuentra su propio código o responde como si fuera ajeno.

#	Pregunta	Respuesta esperada	Señal de alerta
1	"Mostrame dónde manejás un código inexistente. ¿Qué pasa si pongo 999?"	Señala el `if code not in products` y lo corre	No encuentra la línea; busca en toda la pantalla
2	"Corré la venta con cantidad 7. ¿De dónde sale el descuento?"	`qty > 5` → 10 % del subtotal; muestra el cálculo	Describe el descuento pero no puede señalarlo
3	"¿Por qué `read_int` usa `while True`? ¿Qué pasa si lo sacás?"	Tras `ValueError` reintenta; sin el bucle, el programa corta	No asocia el except con el reintento
4	"En tu Parte 1, ¿cómo te diste cuenta del defecto del menú?"	Relata la prueba: eligió 1, salió "Opcion invalida", comparó con	"Lo vi porque estaba raro" sin poder mostrar

#	Pregunta	Respuesta esperada	Señal de alerta
		el código	el síntoma
5	"Cambiale en vivo el descuento a 15 % y mostrame."	Edita `0.10` → `0.15` y corre la prueba de 7 unidades	No sabe qué valor tocar; pide que lo haga el docente
6	"¿De dónde vienen `total_units` y `total_money`? ¿Por qué `global`?"	Acumuladores de la sesión; la función los modifica	No sabe qué contienen ni cuándo crecen
7	"En la Parte 3, ¿qué te dio trabajo del rastreo?"	Explica su ciclo de desvío (si lo hubo) con el fragmento a la vista	No tiene el rastreo escrito o no lo reconoce
8	"Escribí 'abc' en la recarga. ¿Qué pasa y dónde se maneja?"	Mensaje de `read_int` y reintento; señala el `except ValueError`	Espera un crash o no ubica el manejo

Registro: planilla con columna "Verificación oral (2 preguntas)" por alumno.

H. Pauta de devolución

- **Cuándo:** al inicio del encuentro 10 (canon).
- **Cómo:** planilla proyectada con puntajes por criterio, errores frecuentes comentados al grupo (se esperan: comparación de tipos en menú, f-strings sin llaves, borde del descuento en 5) y devolución individual escrita.
- **Registro:** columna "Evaluación U1" sobre 100 + columna de verificación oral.
- **Recuperación:** menos de 60 pts → rehacer el ejercicio en su versión corregida y presentarlo al inicio del encuentro 11. En la recuperación el docente replica la verificación oral sobre el ejercicio rehecho.

Nota final de la propuesta

Este piloto mantiene intactas la estructura del encuentro, el TP con defensa individual, la equivalencia A/B y el umbral de 60. Cambia **qué se evalúa** de la prueba práctica: de producir código desde cero (delegable a un LLM) a leer, diagnosticar, completar y extender código concreto con verificación oral — que es, además, el trabajo real que la materia declara enseñar. Si el docente lo aprueba, la replicación a U2–U4 sigue la misma plantilla con los dominios de cada unidad.